

Guía del Programador

Esta guía explica la estructura actual del código del proyecto "Gestor de Archivos" y dónde localizar los componentes principales.

Estructura general

- **FileManager.Core**
 - Modelos y lógica de acceso al sistema de archivos (interfaces y servicios).
 - Modelos importantes:
 - **FileSystemItem**: representa archivo o carpeta. Propiedades: Name, FullPath, IsDirectory, Size, CreatedAt, ModifiedAt, AccessedAt, Extension, Attributes, DisplaySize, DisplayType.
 - **DriveInfoModel**: información de unidades (Name, Label, DriveType, FileSystem, TotalSize, FreeSpace, DisplayName, DisplayFreeSpace).
- **FileManager.ViewModels**
 - Contiene los ViewModels y la implementación de comandos.
 - **ViewModelBase**: base para notificación `INotifyPropertyChanged` y helper `SetProperty`.
 - **RelayCommand / RelayCommand**: implementación de `ICommand` para enlazar acciones desde XAML.
 - **MainViewModel**: ViewModel principal. Registra **FileExplorerViewModel** y expone `InitializeAsync()` para cargar unidades.
 - **FileExplorerViewModel**: lógica del explorador de archivos. Propiedades y comandos:
 - Propiedades: `CurrentPath`, `SelectedItem`, `IsLoading`, `Items` (`ObservableCollection`), `Drives` (`ObservableCollection`)
 - Comandos: `OpenFileCommand`, `NavigateBackCommand`, `NavigateForwardCommand`, `NavigateToPathCommand`, `NavigateToBreadcrumbCommand`
 - Métodos: `LoadDrivesAsync()`, `NavigateToAsync(path)`, `LoadDirectoryAsync(path)`
- **FileManager.UI**
 - Contiene las vistas WPF y controles personalizados.
 - **App.xaml**: recursos globales, estilos y convertidores.
 - **MainWindow.xaml / MainWindow.xaml.cs**: ventana principal. `DataContext` se asigna en `App.xaml.cs` usando DI.
 - Controles:
 - **TreeViewControl**: control de árbol para navegar carpetas/ unidades (carga bajo demanda).
 - **FileListControl**: control de lista de archivos con doble clic manejado para abrir carpetas.

Bindings y Comandos

- **MainWindow.xaml** está enlazado al **MainViewModel** (`DataContext`). Acceso al **FileExplorerViewModel** desde XAML usa rutas como:
 - `{Binding FileExplorerViewModel.CurrentPath}`
 - `{Binding FileExplorerViewModel.Items}`
 - `{Binding FileExplorerViewModel.Drives}`

- Comandos: {Binding FileExplorerViewModel.NavigateToPathCommand}, {Binding FileExplorerViewModel.OpenFileCommand}, etc.

Cómo añadir comandos nuevos

1. Definir ICommand en FileExplorerViewModel como propiedad pública que devuelva un RelayCommand o RelayCommand.
2. Implementar la lógica en el delegado del comando.
3. Enlazar desde XAML: Command="{Binding FileExplorerViewModel.MyNewCommand}" y, si necesita parámetro: CommandParameter="{Binding ...}".

Inicialización y DI

- En App.xaml.cs se configura un ServiceCollection que registra IFileService, MainViewModel y MainWindow.
- MainWindow.DataContext se asigna al MainViewModel inyectado.
- MainViewModel.InitializeAsync() es llamado en OnStartup para cargar unidades.

Extensiones recomendadas

- Implementar comandos faltantes (NewItemCommand, CutCommand, CopyCommand, PasteCommand, RenameCommand, DeleteCommand) en FileExplorerViewModel y en FileService para realizar las operaciones de I/O.
- Añadir manejo de errores y confirmaciones en operaciones destructivas (Eliminar).
- Añadir tests unitarios para FileService y lógica de ViewModels.

Notas

- Proyecto targeting .NET 8 y C# 12.
- Usar patrones async/await en operaciones de I/O para mantener UI responsiva.

Fin de la guía.